© The Author(s), under exclusive license to APress Media, LLC, part of Springer Nature 2025
S. K. Sahu, *Generative AI-Driven Application Development with Java*
https://doi.org/10.1007/979-8-8688-1609-3_17

17. Talking in Protocols: The MCP Revolution

Satej Kumar Sahu[1]
(1) Berhampur, Odisha, India

The goal of this chapter is to understand what the Model Context Protocol (MCP) protocol is and how it helps to establish a standard protocol on agent AI and tooling.

Introduction

The development of AI took a major step forward when we started creating "tool-augmented agents." Agents gained the ability to take action and make changes in the real world. This was an unprecedented step into the self-development and intelligence explosion era. As generative AI (Gen AI) and large language models (LLMs) continue to evolve into autonomous agents capable of tool invocation and task execution, a new challenge emerges: how these agents communicate with each other and with the broader ecosystem of tools and services. Without a standardized framework, agent interactions remain fragmented, leading to integration inefficiencies and limited scalability.

Chapter 17 explores this crucial issue by introducing the Model Context Protocol—a modular, vendor-neutral communication standard designed to streamline how LLM-powered agents interact with external tools and services. In this chapter, you will gain a comprehensive understanding of how protocols like MCP, Agent Communication Protocol (ACP), Agent-to-Agent Protocol (A2A), and Agent Network Protocol (ANP) form the foundation of interoperable, collaborative AI ecosystems. You will also examine how MCP is practically implemented using modern Java frameworks like Spring Boot and Quarkus, illustrating its role in bridging diverse systems and enabling tool-rich, multi-agent workflows.

By the end of this chapter, developers, architects, and AI practitioners will understand how these protocols collectively advance modularity, standardization, and cross-platform coordination in the AI agent landscape—making it easier to scale, maintain, and evolve intelligent systems.

Protocols in the LLM Ecosystem

As language models increasingly serve as the cognitive engines behind autonomous agents, the challenge facing these systems is shifting from individual capability to collective coordination. While LLM-powered agents excel at interpreting instructions, reasoning through complex tasks, and even invoking external tools, their ability to communicate and collaborate within larger multi-agent ecosystems remains underdeveloped. The lack of standardized communication mechanisms has resulted in fragmented architectures—dominated by proprietary APIs, one-off integrations, and rigid tool bindings. These limitations make scaling and composing intelligent agents across different systems a labor-intensive and brittle endeavor.

To address this gap, a new class of protocols has begun to emerge—designed to enable modularity, interoperability, and dynamic interaction across agents and models. These include MCP, ACP, A2A, and ANP. Each of these protocols plays a distinct role in laying the foundation for a standardized, decentralized agent ecosystem.

Model Context Protocol

MCP is concerned with how LLMs interpret and manage the contextual information necessary to perform tasks effectively. It provides a structured framework for defining the model's working environment—such as memory state, goals, tool access, and user intent—making context portable and reusable across different sessions, tools, or hosts. MCP acts as a bridge between external orchestration systems and the model's internal state, ensuring that LLMs operate with shared assumptions and consistent understanding, regardless of platform or vendor.

Agent Communication Protocol

ACP establishes the foundational syntax and semantics for how agents exchange information. Rather than relying on ad hoc message formats or brittle HTTP endpoints, ACP defines a shared message structure that supports not only command execution and data sharing but also negotiation, delegation, and error handling. This protocol enables a universal communication layer between agents—whether human-controlled, autonomous, or hybrid—unlocking collaborative workflows and coordinated decision-making.

Agent-to-Agent Protocol

While ACP focuses on communication structure, A2A specializes in direct peer interaction among autonomous agents. A2A enables agents to discover, authenticate, and engage

with one another in decentralized or semi-centralized environments. It supports dialogue-based negotiation, mutual goal alignment, task sharing, and status reporting. By embedding social intelligence into machine interactions, A2A transforms isolated agents into networked collaborators capable of forming temporary alliances or persistent coalitions.

Agent Network Protocol

ANP takes a broader, system-level view. It defines how collections of agents—potentially operating across different domains, platforms, or organizations—form scalable and secure networks. ANP governs agent registration, capabilities publishing, routing of tasks across the network, and lifecycle management. Its goal is to enable plug-and-play interoperability between heterogeneous agent systems, much like how IP standardizes communication across the Internet. With ANP, agents become network-addressable entities that can dynamically join or leave collaborative ecosystems without breaking functionality or trust.

Together, these four protocols represent a shift from monolithic, closed-agent systems to composable, cooperative networks of LLM-driven agents. They introduce the necessary abstractions and standards for agents to not just act but to interact—efficiently, securely, and at scale.

What Is MCP Protocol?

The Model Context Protocol—also sometimes known as the Modular Communication Protocol—is a standardized, JSON-RPC framework that facilitates structured communication between LLM clients (such as agents, IDE copilots, or chatbots) and the external components they interact with, including tools, files, APIs, shell commands, and other models.

MCP defines a consistent protocol for sending and receiving messages, invoking tools, handling tasks, and accessing data across distributed, modular systems. It standardizes how agents interact with external services, reducing the need for custom glue code and enabling plug-and-play interoperability. Figure 17-1 emphasizes both the technical mechanism (JSON-RPC over HTTP/TLS) and the step-by-step interaction flow between MCP components.

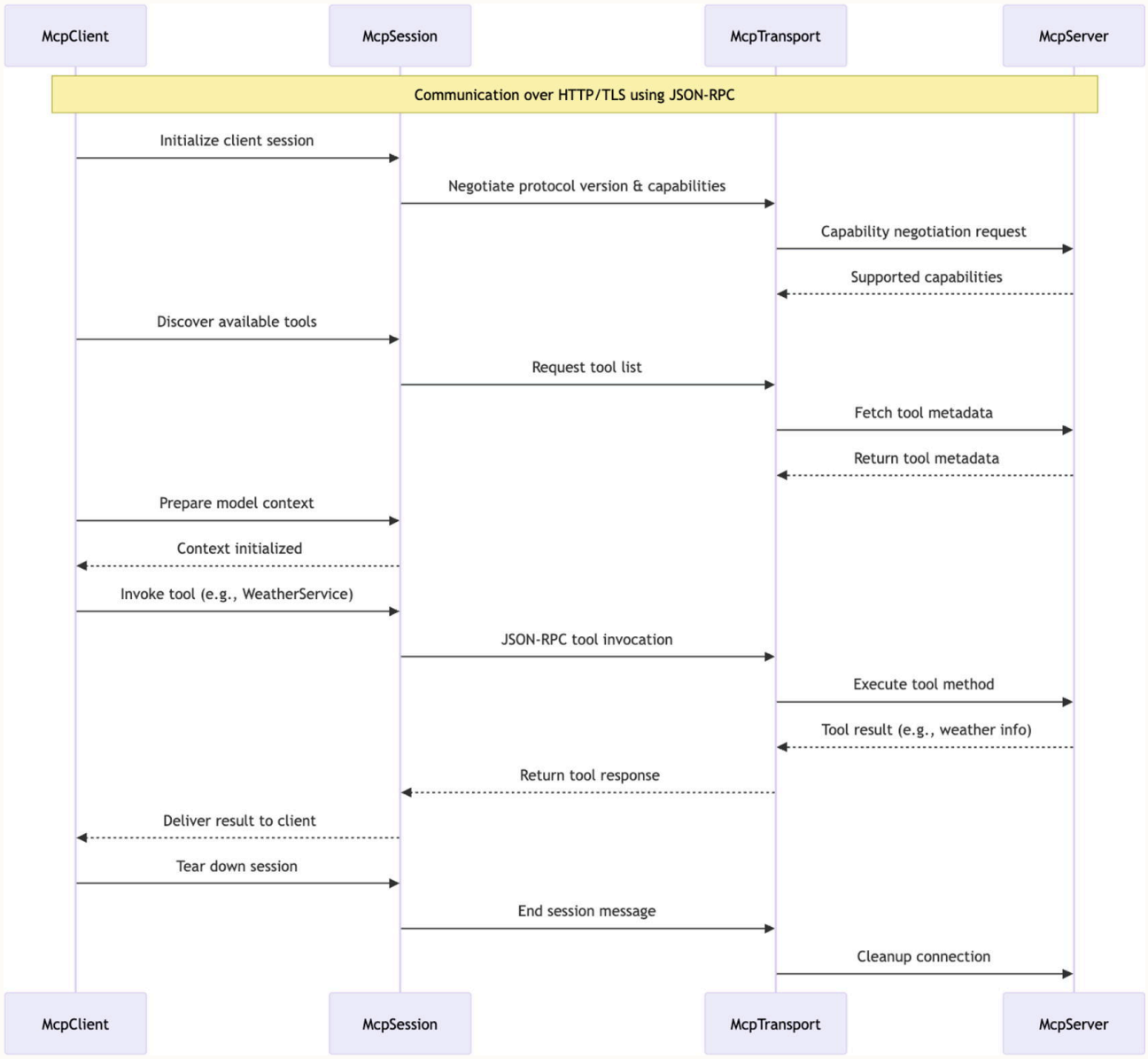


Figure 17-1 MCP communication flow: from client initialization to tool invocation over JSON-RPC

At its core, MCP exposes four key capability types:

- **Resources**: Readable inputs for the model, such as logs, code, database rows, or screenshots
- **Tools**: Callable functions with typed inputs and outputs
- **Prompts**: Reusable prompt templates that can be invoked on demand
- **Sampling**: Host-side code can request completions from the LLM

These capabilities are hosted on servers, which may run locally or in the cloud. A client embedded in a host application maintains a single connection to each server—communicating via stdio or HTTP with Server-Sent Events (SSE). The many-to-many architecture allows agents to mix and match tools or data sources freely, supporting modular development and scalability.

Thanks to its well-defined message schema (typically JSON), MCP enables seamless interoperability between diverse tools and agents, regardless of language or platform. It

also supports advanced features such as task delegation, execution tracking, and result aggregation—making it ideal for scalable AI systems and multi-agent frameworks.

MCP provides a foundational layer for building maintainable, extensible, and interoperable AI-driven applications. For the full specification, see https://modelcontextprotocol.io/specification/2025-03-26.

Role of MCP

The role of MCP is to act as a unifying communication layer between AI agents and the tools or services they use. In increasingly complex AI ecosystems—where agents need to perform tasks, call external APIs, retrieve data, or collaborate with other agents—MCP provides the standardized rules and message formats needed to make these interactions seamless, reliable, and scalable.

One of MCP's key roles is enabling interoperability, allowing different agents and tools—possibly written in different languages or running on different platforms—to work together without custom integration for each pairing. It also facilitates modular system design, where components can be added, removed, or upgraded independently without affecting the overall architecture. Additionally, MCP plays a critical role in task coordination, by structuring how agents issue tool calls, track execution, and handle results. This makes it especially valuable in systems involving orchestration, automation, and agent collaboration. Ultimately, MCP serves as the backbone of agent-tool communication, making intelligent systems more maintainable, consistent, and efficient.

MCP Using Spring AI

MCP is a standardized communication protocol that allows AI models to interact with external tools and resources in a structured and consistent manner. It supports various transport mechanisms, offering flexibility for deployment across different platforms and environments.

The MCP Java SDK delivers a Java-based implementation of this protocol, enabling seamless and standardized communication between AI models and tools. It supports both synchronous and asynchronous messaging patterns to suit diverse application needs.

Building on this foundation, Spring AI MCP integrates the MCP Java SDK with the Spring Boot ecosystem. It offers ready-to-use client and server starters, making it easy to bootstrap AI-powered applications with MCP support using Spring Initializr.

The Java implementation of MCP is structured using a three-layer architecture, as shown in Figure 17-2.

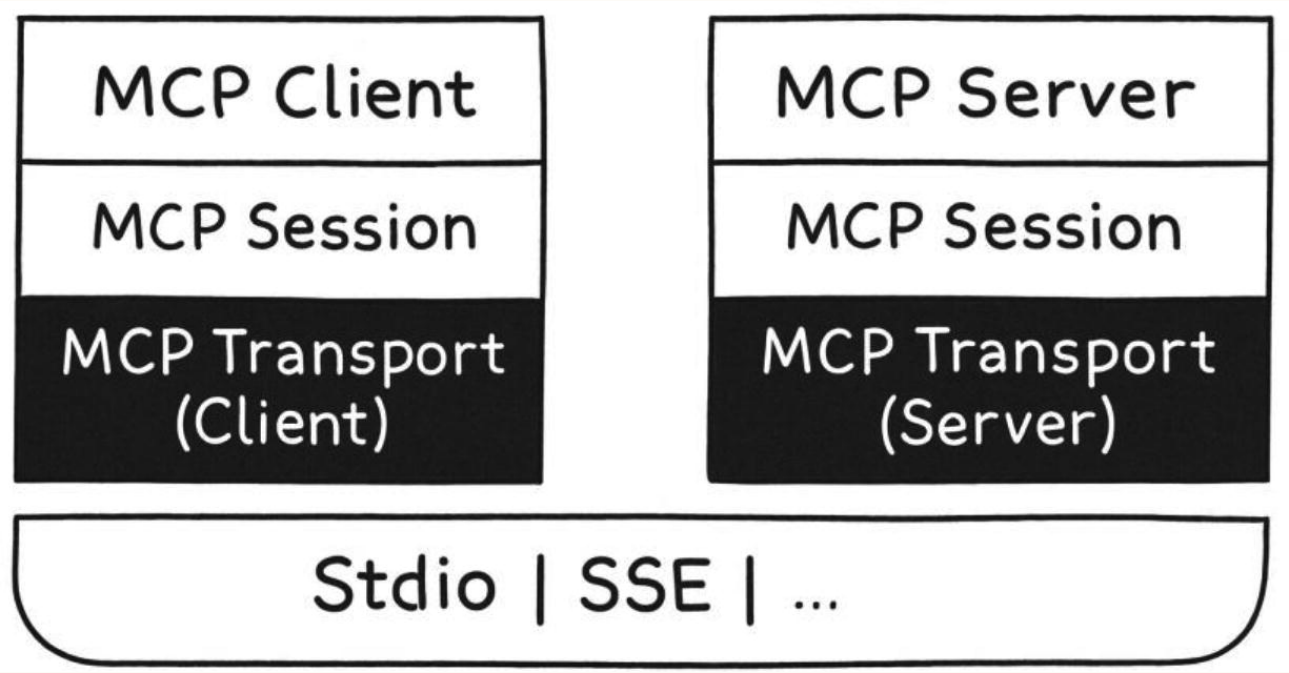


Figure 17-2 Three-tier architecture of MCP in Java

Here is more about the layers:

1. **Client/server layer:** McpClient manages client-side interactions, while McpServer handles protocol operations on the server side. Both components rely on McpSession to manage their communication flow.
2. **Session layer (McpSession):** This layer is responsible for maintaining communication state and managing interaction patterns, typically implemented through DefaultMcpSession.
3. **Transport layer (McpTransport):** This layer takes care of serializing and deserializing JSON-RPC messages and supports multiple transport mechanisms for flexible integration.

MCP Client

The MCP client, as depicted in Figure 17-3, is a central component within the MCP architecture, responsible for initiating and managing communication with MCP servers. It implements the client-side functionality of the protocol and handles a range of critical tasks, including:

- Negotiating protocol versions to ensure server compatibility
- Determining supported features through capability negotiation
- Managing message transport using JSON-RPC
- Discovering and executing tools
- Accessing and managing resources

- Interacting with the prompt system

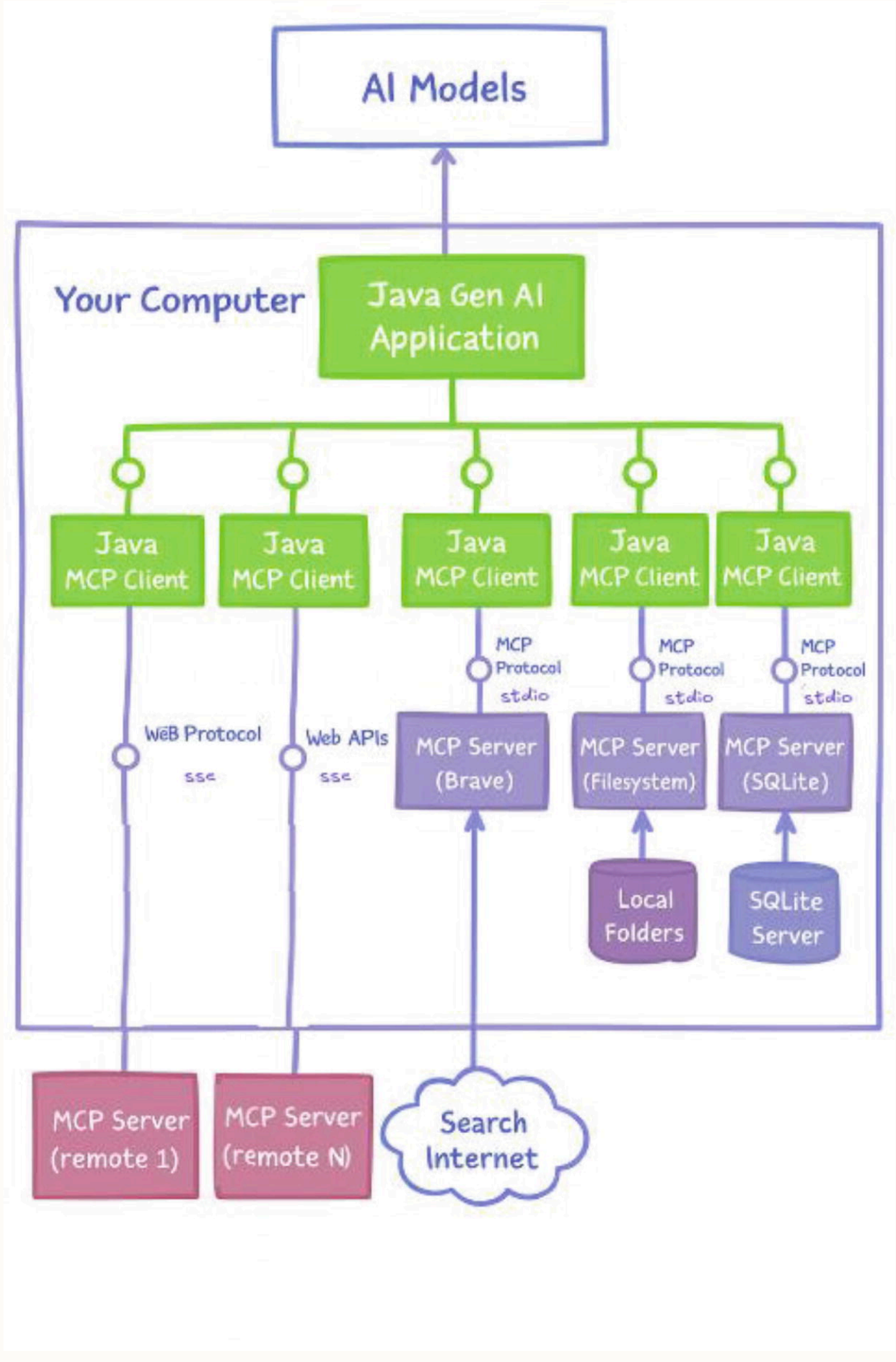


Figure 17-3 MCP client

Additionally, the MCP Client supports optional features such as:

- Roots management
- Sampling operations

It is designed to support both synchronous and asynchronous communication patterns and offers flexible transport mechanisms, including:

- Stdio-based transport for process-level communication
- Java HttpClient-based Server-Sent Events (SSE) transport
- WebFlux SSE client transport for reactive HTTP streaming

MCP Server

The MCP server, as shown in Figure 17-4, is a core component of the MCP architecture, designed to provide tools, resources, and capabilities to connected clients. It implements the server-side logic of the protocol and is responsible for the following:

- Executing protocol operations on the server
- Exposing tools for discovery and invocation
- Managing resources via URI-based access
- Providing and handling prompt templates
- Negotiating capabilities with clients
- Delivering structured logging and event notifications

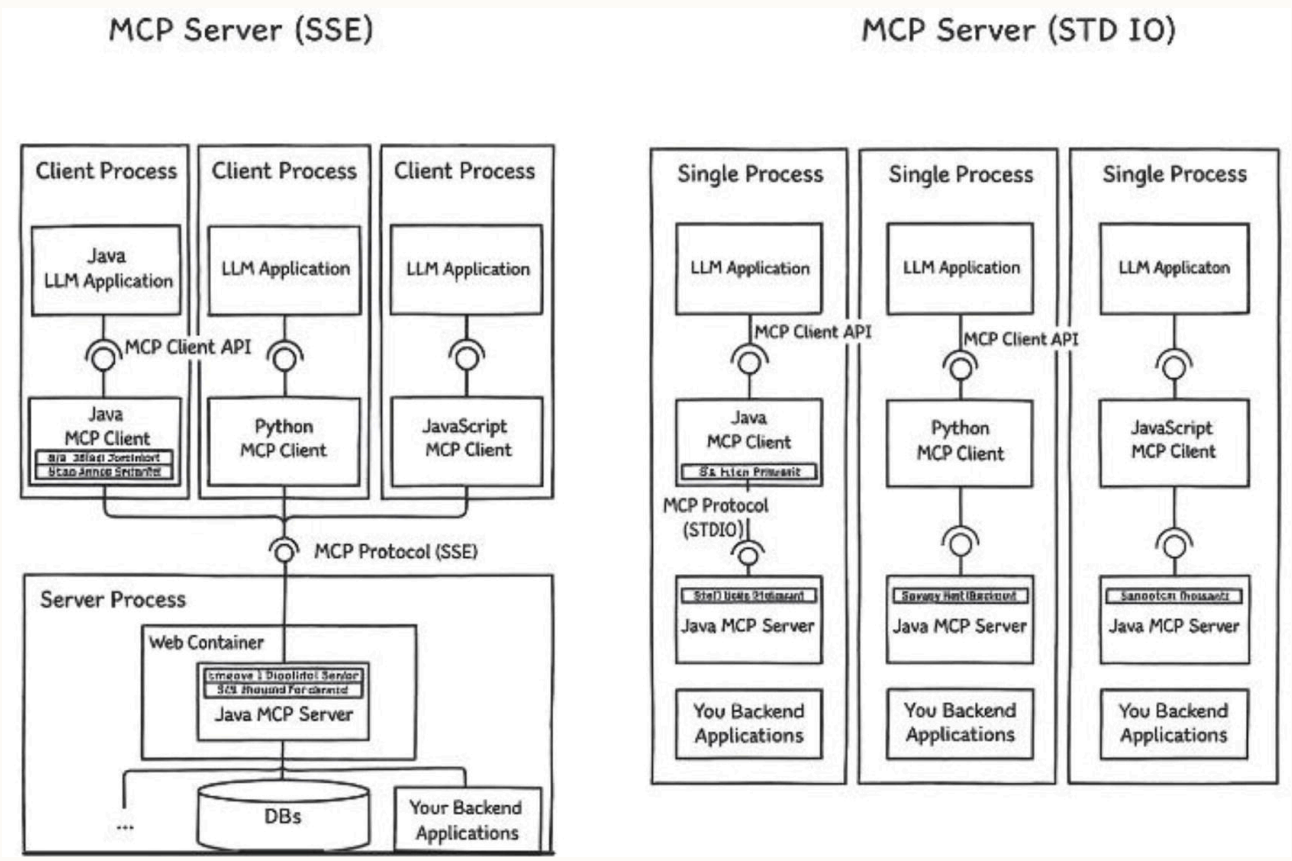


Figure 17-4 MCP server

In addition to supporting both synchronous and asynchronous APIs, the MCP server is built to efficiently handle multiple concurrent client connections.

It also offers various transport implementations, including:

- Stdio-based transport for communication with local processes
- Servlet-based Server-Sent Events (SSE) transport
- WebFlux SSE server transport for reactive HTTP streaming
- WebMVC SSE server transport for servlet-based HTTP streaming

Spring AI MCP Integration

Spring AI supports seamless integration with MCP through a set of dedicated Spring Boot starters, categorized into client and server modules:

Client Starters:

- **spring-ai-starter-mcp-client**: Core client starter that includes support for STDIO and HTTP-based SSE communication
- **spring-ai-starter-mcp-client-webflux**: Provides a WebFlux-based SSE transport implementation for reactive client communication

Server Starters:

- **spring-ai-starter-mcp-server**: Core server starter with STDIO transport support
- **spring-ai-starter-mcp-server-webmvc**: Implements a Spring MVC-based SSE transport for servlet-driven environments
- **spring-ai-starter-mcp-server-webflux**: Offers a WebFlux-based SSE transport for fully reactive server communication

We will build two projects. One sets up the Spring AI–based MCP server, and the other is a Spring AI-based MCP client that can connect to the MCP server and invoke tools exposed by it.

Capabilities of the MCP Server in Spring AI

The Spring AI integration with MCP empowers developers to build AI-enabled applications that expose well-structured and interoperable capabilities to connected clients. The MCP Server Boot Starter plays a central role in this architecture by enabling servers

to advertise and manage a variety of functionalities such as tools, resources, prompts, completions, and root change handlers. These capabilities are automatically adapted to either synchronous or asynchronous modes, depending on the configuration of the server.

Tool Exposure and Management

One of the primary features of the MCP server is its ability to expose *tools* that language models can invoke during execution. Tools are implemented as Spring beans and are automatically discovered and registered by the framework. The auto-configuration process ensures that tools with duplicate names are deduplicated, keeping only the first registered instance. The MCP framework also supports *tool context*, which allows passing contextual metadata—like logging or interaction state—during tool invocation. This context-aware execution makes tool behavior more dynamic and responsive.

For this chapter, the implementation focuses specifically on this capability. I will demonstrate how a tool (in this case, a basic weather information provider) can be integrated into a Spring Boot-based MCP server and accessed by a remote MCP client. This example lays the foundation for more complex tool interactions and modular AI service architectures.

Resource Management

MCP also enables standardized *resource management*, allowing servers to expose both static and dynamic data to clients. These resources can include configuration files, status information, system metrics, or any structured data relevant to the client's tasks. Change notifications are supported, helping clients stay in sync with updated content. Resource specifications are defined as Spring beans and automatically registered with the server.

Prompt Templates

With *prompt management*, the MCP server can deliver templated prompts that clients can use for consistent, parameterized interactions. This includes support for versioning, argument validation, and change tracking. Prompts are particularly useful in multi-agent settings where uniform communication or behavior patterns are required.

Completions

The *completion management* capability allows the server to offer AI-powered suggestions or auto-completions to connected clients. These could range from code snippets to sen-

tence continuations or query refinements. Completion services can be configured as either synchronous or asynchronous, depending on application needs.

Root Change Monitoring

MCP servers can monitor and react to *root changes*—structural updates that impact available tools, resources, or system configurations. When such changes occur, the server can notify clients, ensuring that they are working with the latest capabilities. Support for this feature is optional but beneficial in dynamic or distributed environments.

In this chapter, we demonstrate only the tooling capability of the MCP server using Spring AI, as it forms the cornerstone of practical LLM integration—enabling modular, callable functionality that can be independently developed, tested, and deployed.

Creating Spring AI-Based MCP Server

The goal for this project is to create an MCP server that exposes a tooling layer. You will keep the tool simple for now by hard-coding the returned value. As you have learned in previous chapters, you can replace the tool with any search engine or weather API and make it more usable as per your use case. Figure 17-6 illustrates how different teams or organizations can independently manage and expose their own MCP servers, enabling modular and scalable AI tool integration.

Add one dependency, as shown in Figure 17-5, which is the Spring AI MCP server package. Let's download and unzip it at the directory of your choice. You will make one change to the build.gradle file. Instead of spring-ai-starter-mcp-server, youwill use the spring-ai-starter-mcp-server-webmvc starter package since it comes with WebMVC transport integration.

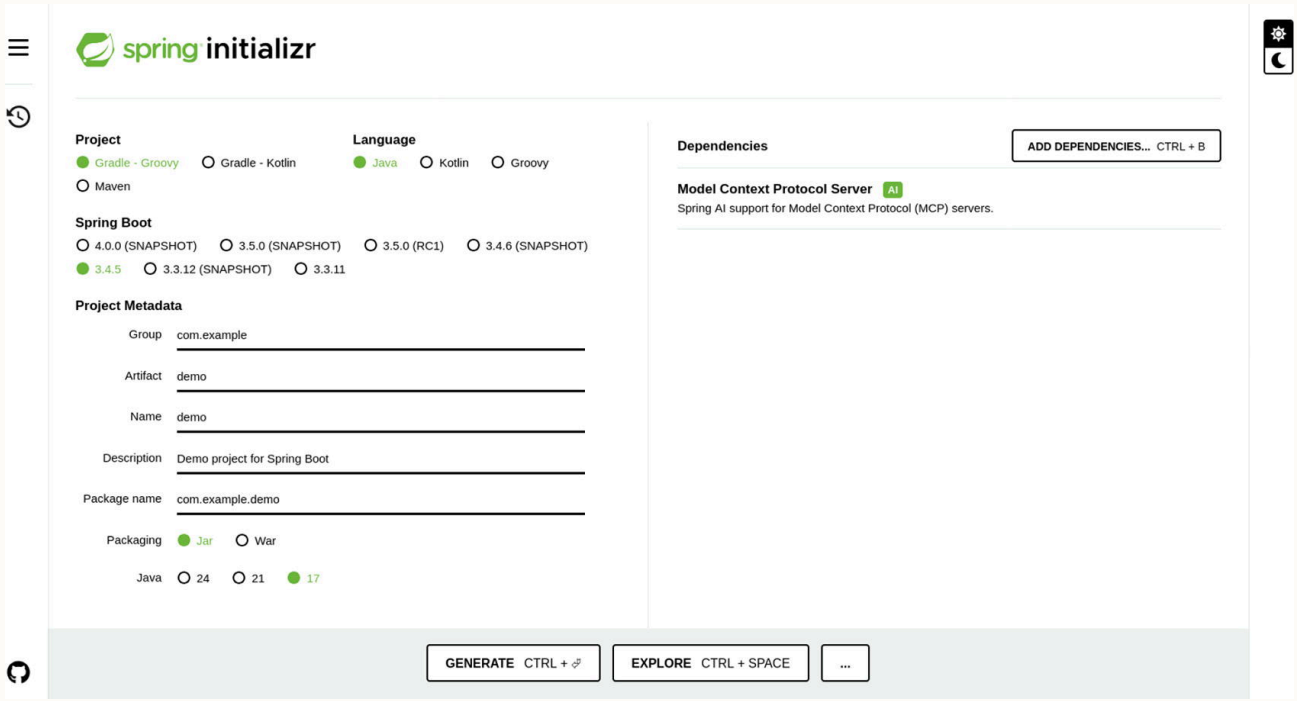


Figure 17-5 Distributed MCP architecture: independent tooling services managed by separate teams

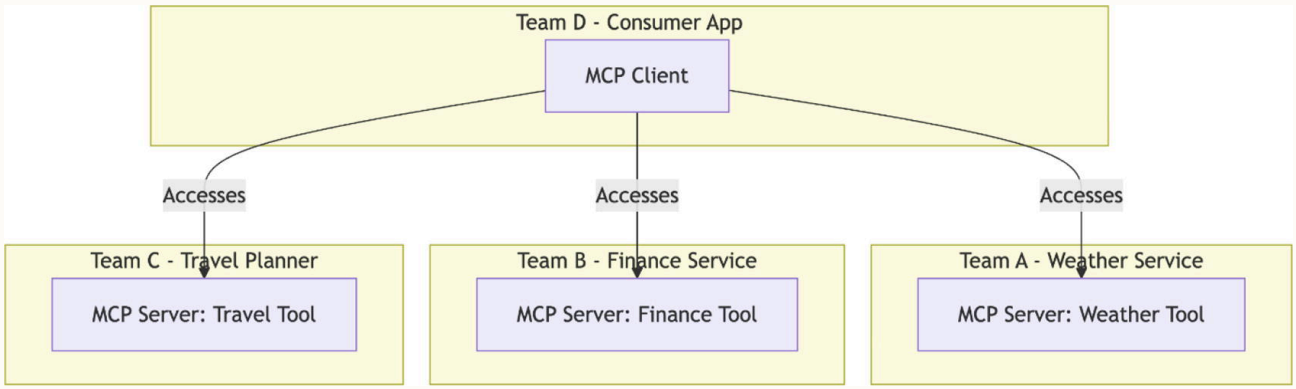


Figure 17-6 Spring Initializr for MCP server

File: ./build.gradle

```
. . .
dependencies {
    implementation 'org.springframework.ai:spring-ai-starter-mcp-server-webmvc'
    testImplementation 'org.springframework.boot:spring-boot-starter-test'
    testRuntimeOnly 'org.junit.platform:junit-platform-launcher'
}
. . .
```

If you start the application now, you should see output similar to the following:

```
2025-05-20T20:07:40.574Z  INFO 2647 --- [demo] [main] com.example.demo.Demo
Application        : Started DemoApplication in 4.504 seconds (process running for 6.07
7)
```

Next, you will update a few properties in the application.properties file for the MCP Server setup.

File: ./src/main/resources/application.properties

```
spring.application.name=demo
spring.ai.mcp.server.name=my-weather-server
spring.ai.mcp.server.version=0.0.1
# Using spring-ai-starter-mcp-server-webmvc
spring.ai.mcp.server.type=sync
spring.ai.mcp.server.instructions="This server provides weather information for place when provided with a longitude and latitude"
spring.ai.mcp.server.sse-endpoint=/sse
spring.ai.mcp.server.capabilities.completion=true
spring.ai.mcp.server.capabilities.prompt=true
spring.ai.mcp.server.capabilities.resource=true
spring.ai.mcp.server.capabilities.tool=true
```

In the previous application.properties file, all properties except the instructions one has default values. These are the different capabilities you are exposing off of the MCP server including completion, prompt, resource, and tools. They can be ignored and have been shared for informational purposes.

You will create a WeatherService where you will define the tool like you did previously by using the weather API service, only in this case you will hard-code the response for simplicity's sake.

File: ./src/main/java/com/example/demo/service/WeatherService.java

```
package com.example.demo.service;
import org.springframework.ai.tool.annotation.Tool;
import org.springframework.stereotype.Service;
@Service
public class WeatherService {
    @Tool(description = "Get weather details for a specific latitude/longitude")
    public String getWeatherInformationByLocation(double longitude, double latitude)
{
        // A custom logic can be replaced here with a call to a weather API service.
        return "The weather is expected to be pleasant with some expectation of snow in the evening";
    }
}
```

The final task in the MCP server is to register this tool as part of the tool set in the application file.

File: ./src/main/java/com/example/demo/DemoApplication.java

```
package com.example.demo;
import com.example.demo.service.WeatherService;
import org.springframework.ai.tool.ToolCallbackProvider;
import org.springframework.ai.tool.method.MethodToolCallbackProvider;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.annotation.Bean;
@SpringBootApplication
public class DemoApplication {
  public static void main(String args) {
      SpringApplication.run(DemoApplication.class, args);
   }
  @Bean
   public ToolCallbackProvider weatherTools(WeatherService weatherService) {
      return MethodToolCallbackProvider.builder().toolObjects(weatherService).buil
d();
   }
}
```

Let's start our application and see the logs in action:

```
2025-05-20T20:29:12.427Z  INFO 4079 --- [demo] [main] o.a.c.c.C.[Tomcat].[localho
st].      : Initializing Spring embedded WebApplicationContext
2025-05-20T20:29:12.428Z  INFO 4079 --- [demo] [main] w.s.c.ServletWebServerA
pplicationContext : Root WebApplicationContext: initialization completed in 1293 ms
2025-05-20T20:29:13.119Z  INFO 4079 --- [demo] [main] o.s.a.m.s.a.McpServerAuto
Configuration   : Enable tools capabilities, notification: true
2025-05-20T20:29:13.220Z  INFO 4079 --- [demo] [main] o.s.a.m.s.a.McpServerAut
oConfiguration   : Registered tools: 1
2025-05-20T20:29:13.220Z  INFO 4079 --- [demo] [main] o.s.a.m.s.a.McpServerAut
oConfiguration   : Enable resources capabilities, notification: true
2025-05-20T20:29:13.221Z  INFO 4079 --- [demo] [main] o.s.a.m.s.a.McpServerAuto
Configuration   : Enable prompts capabilities, notification: true
2025-05-20T20:29:13.222Z  INFO 4079 --- [demo] [main] o.s.a.m.s.a.McpServerAut
oConfiguration   : Enable completions capabilities
2025-05-20T20:29:13.338Z  INFO 4079 --- [demo] [main] o.s.b.w.embedded.tomca
t.TomcatWebServer  : Tomcat started on port 8080 (http) with context path '/'
```

```
2025-05-20T20:29:13.353Z  INFO 4079 --- [demo] [main] com.example.demo.Demo
Application      : Started DemoApplication in 2.925 seconds (process running for 3.8
86)
2025-05-20T20:29:58.952Z  INFO 4079 --- [demo] [nio-8080-exec-1] o.a.c.c.C.[Tom
cat].[localhost].[/]      : Initializing Spring DispatcherServlet 'dispatcherServlet'
2025-05-20T20:29:58.953Z  INFO 4079 --- [demo] [nio-8080-exec-1] o.s.web.servle
t.DispatcherServlet       : Initializing Servlet 'dispatcherServlet'
2025-05-20T20:29:58.955Z  INFO 4079 --- [demo] [nio-8080-exec-1] o.s.web.servle
t.DispatcherServlet       : Completed initialization in 1 ms
```

In the previous logs, you can see information relevant to the capabilities available to the MCP server and that one tool has been registered for the tools layer. We have been used to developing microservice based applications. Similar to it, this component project could be understood as having been developed by one team or a different organization altogether. This will be exposed as tool service, helping to keep it modular similar to microservices. There must be an authentication layer for every request coming through. For simplicity's sake, we have ignored it for this use case.

Creating a Spring AI-Based MCP Client

Playing the role of a team member in another team, the application needs to use the weather tool service to get weather details. Let's build the MCP client project starting with the Spring Initializr website dependencies, as shown in Figure 17-7.

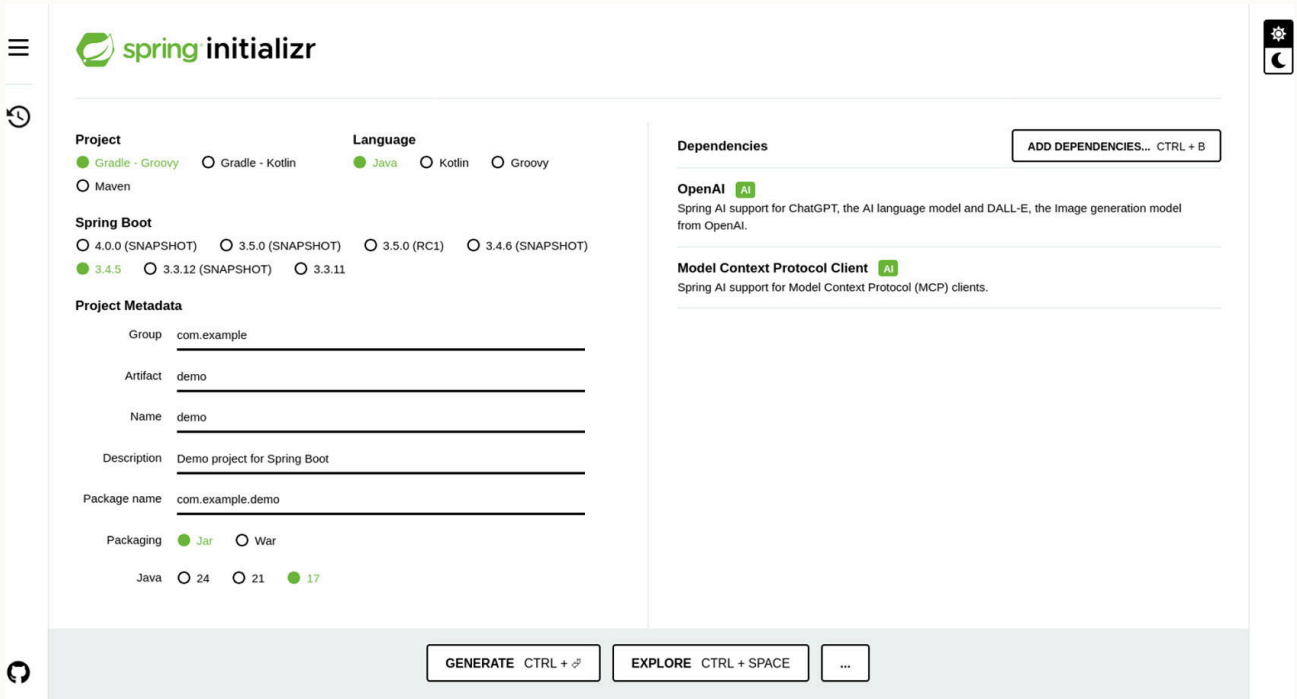


Figure 17-7 Spring Initializr dependencies for MCP client

Download the zip file and unzip it in the directory of your choice. You will start with setting a few properties in the application.properties file to enable some basic configuration

setups relevant to the MCP client and establish its connection to the MCP server.

File: ./src/main/resources/application.properties

```
spring.application.name=demo
spring.main.web-application-type=none
spring.ai.openai.api-key=YOUR_OPENAI_API_KEY
spring.ai.mcp.client.toolcallback.enabled=true
spring.ai.mcp.client.sse.connections.server1.url=http://localhost:8080
logging.level.io.modelcontextprotocol.client=WARN
logging.level.io.modelcontextprotocol.spec=WARN
ai.user.input=What is the climate of Ladakh in India which has latitude and longitude of 34.2268° N, 77.5619° E?
```

As we can see, here you have set up the toolcallback capability to be enabled for the MCP client and set the MCP server endpoint to which the MCP client has to connect to use the tool capability of the MCP server. There could be more than one server. For our example at hand and for simplicity, I have chosen one. Then you set some logging properties relevant to MCP protocol for WARN level logging. Finally, you set a default question you will be asking the LLM model when the application boots up.

With the configuration out of our way, you will use the CommandLineRunner pattern to invoke a method when the application boots up, as shown here:

File: ./src/main/java/com/example/demo/DemoApplication.java

```
package com.example.demo;
import org.springframework.ai.chat.client.ChatClient;
import org.springframework.ai.tool.ToolCallbackProvider;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.ConfigurableApplicationContext;
import org.springframework.context.annotation.Bean;
@SpringBootApplication
public class DemoApplication {
    @Value("${ai.user.input}")
    private String userInput;
    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
    }
    @Bean
```

```
    public CommandLineRunner determineWeather(ChatClient.Builder chatClientBuilde
r, ToolCallbackProvider tools,
                                ConfigurableApplicationContext context) {
        return args -> {
            ChatClient chatClient = chatClientBuilder
                    .defaultToolCallbacks(tools)
                    .build();
            System.out.println("\n>>> UserQuestion: " + userInput);
            System.out.println("\n>>> LLM Answer: " + chatClient.prompt(userInput).call().
content());
            context.close();
        };
    }
}
```

In the previous code, you can see that you are using the runtime-injected instances of the ChatClient.Builder API, which will be used to instantiate the ChatClient API. The tools argument contains the tool interface-specific information that is shared by you MCP server and gathered by the MCP client and shared with the LLM model API. Finally, you also have a reference to the MCP context and will use it to close the connection to the MCP server.

Let's start the MCP client app and see the logs in action:

```
2025-05-20T21:09:52.975Z  INFO 5801 --- [demo] [main] com.example.demo.DemoA
pplication       : Started DemoApplication in 2.666 seconds (process running for 3.386)
>>> UserQuestion: What is the climate of Ladakh in India which has latitude and longit
ude of 34.2268° N, 77.5619° E?
>>> LLM Answer: The climate of Ladakh is currently pleasant, with some expectation o
f snow in the evening.
Process finished with exit code 0
```

This is cool. You can see that the MCP client connected with the MCP server to invoke the relevant tool. This is an interesting example of multiple teams collaborating and maintaining a tooling layer that is complex without having to see the underlying details of it. The following is the log on the MCP server side, which shows a request had been made to it:

```
2025-05-20T21:09:52.456Z  INFO 5328 --- [demo] [nio-8080-exec-9] i.m.server.Mc
pAsyncServer              : Client initialize request - Protocol: 2024-11-05, Capabilities: Cli
entCapabilities[experimental=null, roots=null, sampling=null], Info: Implementation[n
ame=spring-ai-mcp-client - server1, version=1.0.0]
```

Integrating MCP with Quarkus

Now that you have learned about MCP clients and servers and built your first application, you will try to reuse some of the practices learned with Quarkus. You will restructure the weather application built previously for Spring AI. So, the goal is to build an MCP server that provides tools for fetching weather details. The server will expose one tool called getWeatherDetailUsingLocation.

Let's now set up the project. Head over to https://code.quarkus.io/ and select the following package dependencies:

- rest-client-jackson
- qute
- mcp-server-stdio

As you follow along, you will see the usage of each of these extension packages. Figure 17-8 shows the listing at https://code.quarkus.io/.

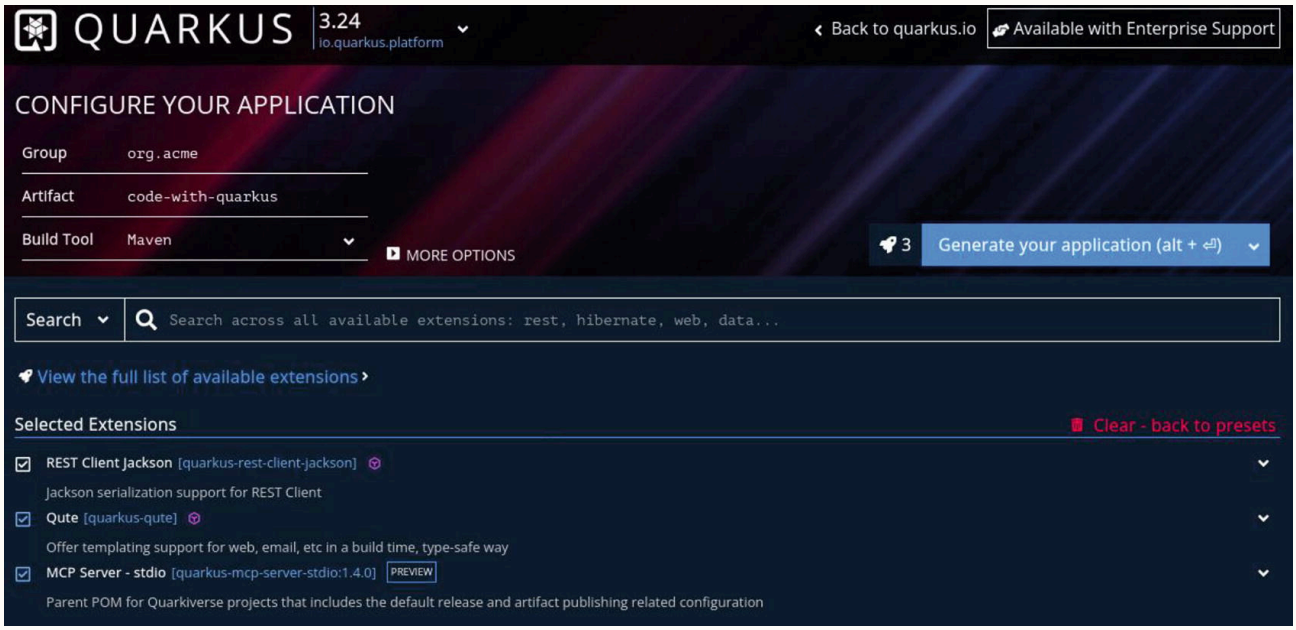


Figure 17-8 MCP implementation-specific dependencies in Quarkus

Download the zip and unzip it at the location of your choice. From your command line, run the following commands to make sure everything works fine. Alternatively, you can boot over your favorite IDE too.

Here are the prerequisites:

1.Java 21+

```
$ ./mvnw clean package
```

```
$ ./mvnw quarkus:dev
```

If everything goes well, you should see output similar to Figure 17-9 and Figure 17-10.

```
Listening for transport dt_socket at address: 5005
__  ____  __  _____   ___  __ ____  ______
 --/ __ \/ / / / _ | / _ \/ //_/ / / / __/
 -/ /_/ / /_/ / __ |/ , _/ ,< / /_/ /\ \
--\___\_\____/_/ |_/_/|_/_/|_|\____/___/
2025-07-23 19:44:19,469 INFO  [io.quarkus] (Quarkus Main Thread) code-with-quarkus 1.0.0-SNAPSHOT on JVM (powe
red by Quarkus 3.24.5) started in 4.281s. Listening on: http://localhost:8080

2025-07-23 19:44:19,475 INFO  [io.quarkus] (Quarkus Main Thread) Profile dev activated. Live Coding activated.
2025-07-23 19:44:19,476 INFO  [io.quarkus] (Quarkus Main Thread) Installed features: [cdi, mcp-server-stdio, q
uute, rest, rest-client, rest-client-jackson, smallrye-context-propagation, vertx]

--
Tests paused
Press [e] to edit command line args (currently ''), [r] to resume testing, [o] Toggle test output, [:] for the
 terminal, [h] for more options>
```

Figure 17-9 Successful output message on first boot of Quarkus

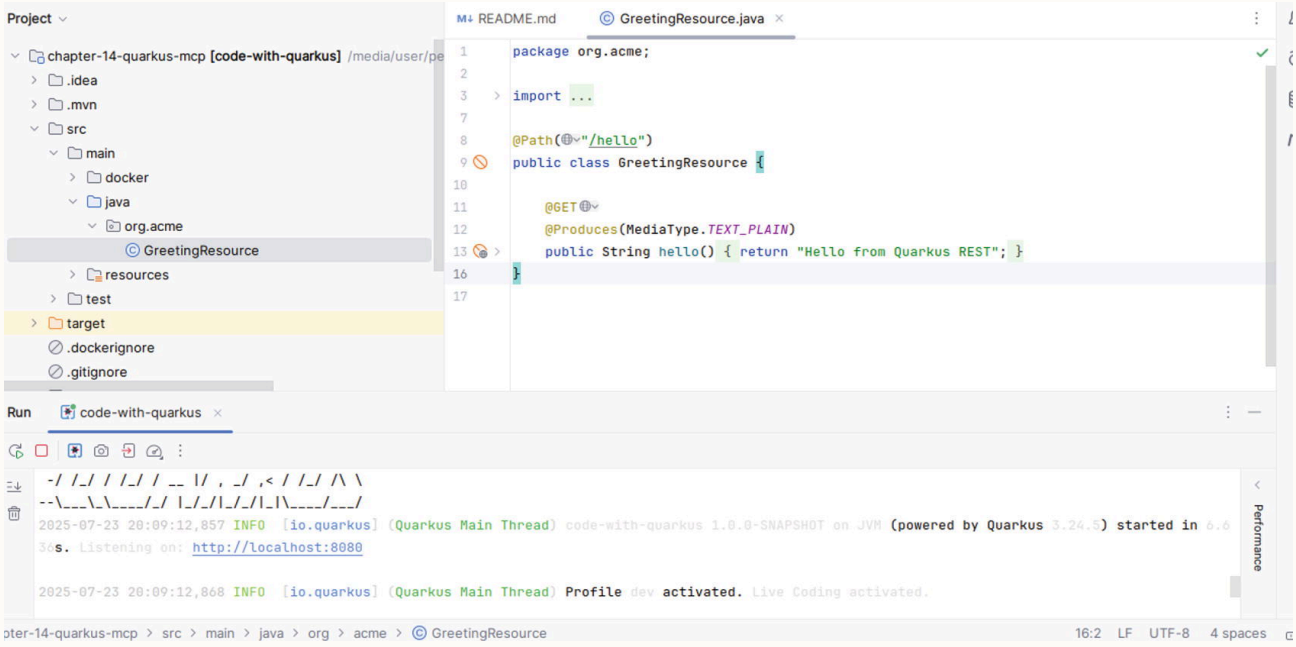


Figure 17-10 Intellij IDE Output

The next task is to create a Java class to return weather details for a location based on latitude and longitude information. I will not go into the details of integrating a third-party API service to get weather details for fetching the information based on location. There are many services that can be used. The focus is on building a tool that returns weather information in string format. You will mock the method for the time being so as to keep our focus on building the MCP server with a weather tool.

Create a Java file called Weather.java with the following content that returns a simple string about the climate in weather at some point. As you can see, you are creating a method named getWeatherDetails that has two inputs and a single String return type.

This method needs to be exposed as a tool from this MCP server. To do so, let's add a tool annotator with a description along with ToolArg to explicitly specify the tool's usage and its arguments.

File: src/main/java/org/acme/Weather.java

```
package org.acme;
import io.quarkiverse.mcp.server.Tool;
import io.quarkiverse.mcp.server.ToolArg;
public class Weather {
   @Tool(description = "Get weather details for a location based on latitude and longit
ude.")
   String getWeatherDetails(@ToolArg(description = "Latitude of the location") doubl
e latitude,
                       @ToolArg(description = "Longitude of the location") double longitud
e) {
      // Make any api call to a third party weather api service to get weather details.
      return "The weather is pleasant sunny with a slight prediction of rain in Bangalor
e.";
   }
}
```

you will now begin preparations to run the server. To simplify development, you will deploy it as an uber-jar. This helps in general to run along with mvn install and enables you to publish and share the JAR with others and the Maven repository. You can do this by using the following property in the application.properties file.

```
quarkus.package.jar.type=uber-jar
```

At this step, it is also pragmatic to enable file logging since input/output is reserved for the MCP protocol, and we would not be able to see any logs by default. To do this, use the following properties in the application.properties file:

```
quarkus.log.file.enable=true
quarkus.log.file.path=weather-mcp-server.log
```

The application.properties file should look like the following:

File: src/main/resources/application.properties

```
quarkus.package.jar.type=uber-jar
quarkus.log.file.enable=true
```

```
quarkus.log.file.path=weather-mcp-server.log
```

There are a few ways you can start the MCP server:

1. Through IntelliJ or any editor specific run.
   Output:

```
2025-07-27 21:37:22,724 INFO  [io.quarkus] (Quarkus Main Thread) code-with-quarkus 1.0.0-SNAPSHOT on JVM (powered by Quarkus 3.24.5) started in 4.222s. Listening on: http://localhost:8080
2025-07-27 21:37:22,730 INFO  [io.quarkus] (Quarkus Main Thread) Profile dev activated. Live Coding activated.
2025-07-27 21:37:22,731 INFO  [io.quarkus] (Quarkus Main Thread) Installed features: [cdi, mcp-server-stdio, qute, rest, rest-client, rest-client-jackson, smallrye-context-propagation, vertx]
```

2. Using the command line, the first step is to install all the Maven package dependencies.

```
./mvnw install
```

Once all the packages are installed, you can start the MCP server using one of the following commands:

- **Using jbang**: jbang --quiet org.acme:code-with-quarkus:1.0.0-SNAPSHOT:runner
- **Using java:** java -jar target/quarkus-app/quarkus-run.jar

Here is the output:

```
2025-07-27 21:39:37,970 INFO  [io.quarkus] (main) code-with-quarkus 1.0.0-SNAPSHOT on JVM (powered by Quarkus 3.24.5) started in 1.896s. Listening on: http://0.0.0.0:8080
2025-07-27 21:39:37,986 INFO  [io.quarkus] (main) Profile prod activated.
2025-07-27 21:39:37,988 INFO  [io.quarkus] (main) Installed features: [cdi, mcp-server-stdio, qute, rest, rest-client, rest-client-jackson, smallrye-context-propagation, vertx]
```

This is all good, but how do we evaluate our tool? There is a handy tool using the MCP inspector from the npm package. As a prerequisite, make sure you have npm and node installed with the latest LTS version. Run the following command to install the package and start it:

```
npx @modelcontextprotocol/inspector
```

Output:

```
Starting MCP inspector...
⚙ Proxy server listening on localhost:6277
🔑 Session token: c2b2d4defea8e2a9f4f4fc8cf252f243aef3aa0401225
745ea07b22629599347
   Use this token to authenticate requests or set DANGEROUSLY_OMI
T_AUTH=true to disable auth
🚀 MCP Inspector is up and running at:
   http://localhost:6274/?MCP_PROXY_AUTH_TOKEN=c2b2d4defea8e2a9f4f4f
c8cf252f243aef3aa0401225745ea07b22629599347
🌐 Opening browser...
```

It then opens a browser tab with the content shown in Figure 17-11.

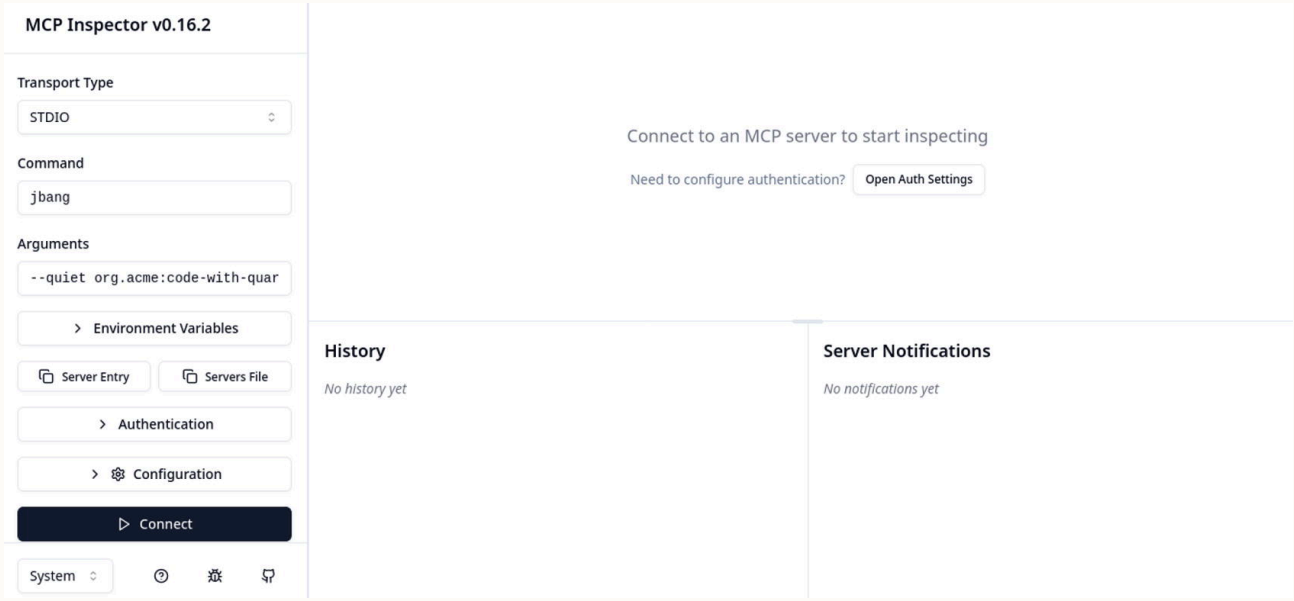


Figure 17-11 MCP Inspector v0.16.2

This starts a local web server where you can test your MCP server. MCP Inspector uses the command for starting your MCP server and then initiates a connection with it as an MCP client. Let's try it and explore your MCP server.

In the side window shown in Figure 17-12, fill in the details for the Command value as jbang and arguments as --quiet org.acme:code-with-quarkus:1.0.0-SNAPSHOT:runner and click the Connect button.

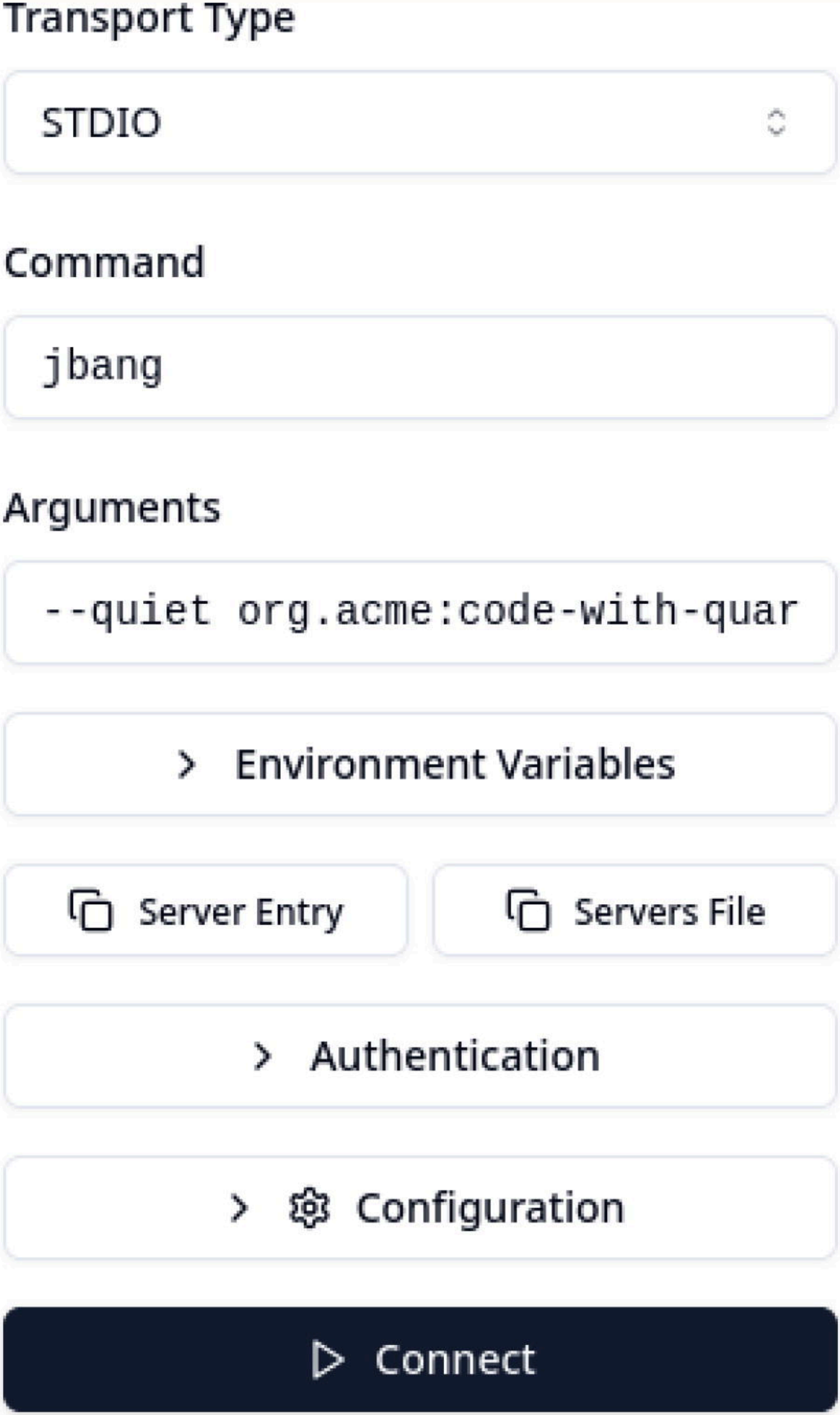


Figure 17-12 MCP Inspector Connect command configuration

Once you connect, the user interface shows a message called Connected with a green signal just below the connect button. In the history panel, you can see an initialize request, as shown in Figure 17-13.

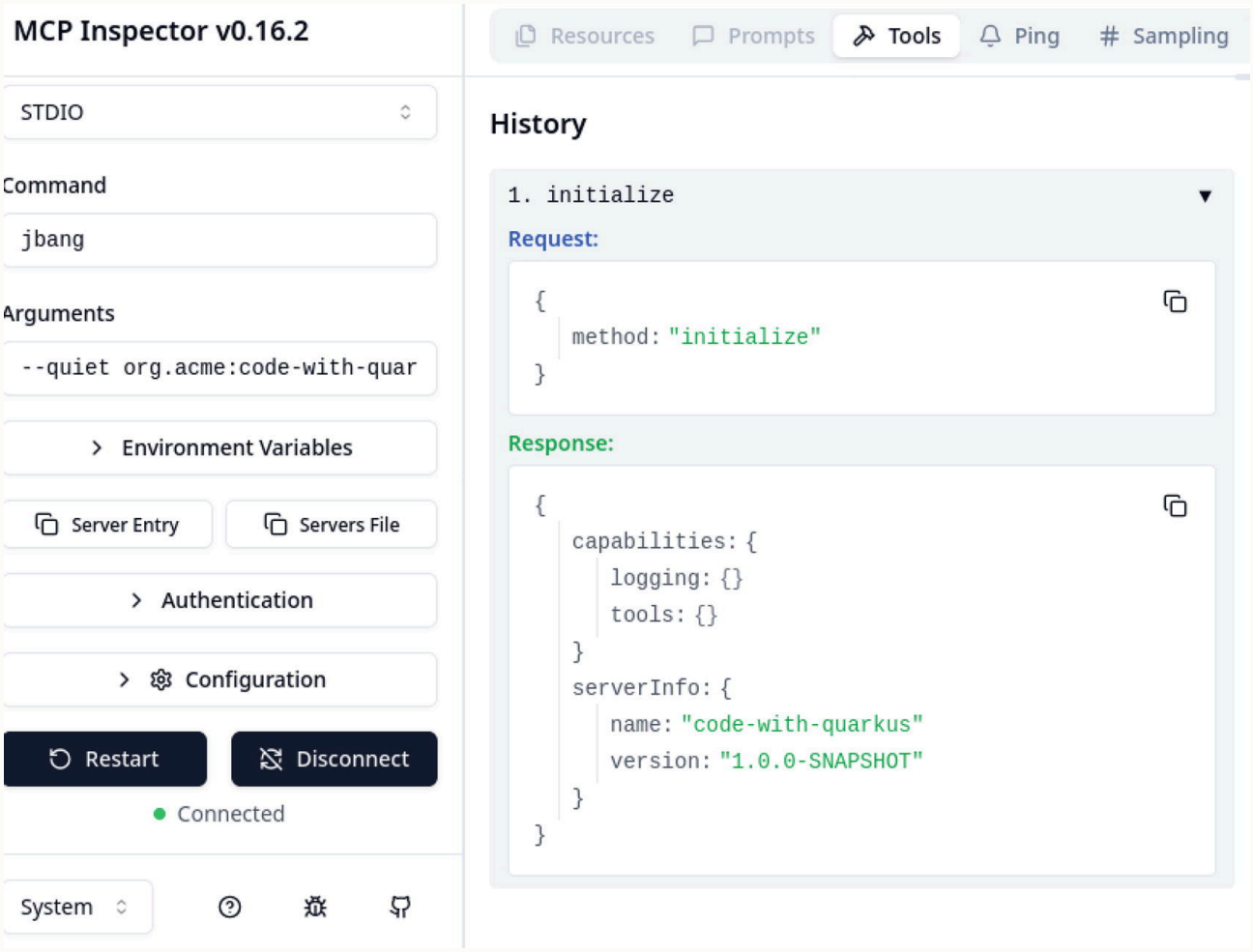


Figure 17-13 MCP Inspector connected to MCP server

The initialize request to the MCP server received a response of the capabilities and serverInfo information.

Correspondingly, you can see the following output in the terminal where you started the MCP connector. This tells you that the MCP connector has started the MCP server and then connected to it through an MCP client.

```
New STDIO connection request
Query parameters: {"command":"jbang","args":"--quiet org.acme:code-with-quarkus:
1.0.0-SNAPSHOT:runner","env":"{\"HOME\":\"/home/user\",\"LOGNAME\":\"user
\",\"PATH\":\"/home/user/.npm/_npx/5a9d879542beca3a/node_modules/.bin:/path/c
hapter-14-quarkus-mcp/node_modules/.bin:/path/node_modules/.bin:/media/user/no
de_modules/.bin:/media/node_modules/.bin:/node_modules/.bin:/home/user/.nvm/ver
sions/node/v22.17.1/lib/node_modules/npm/node_modules/@npmcli/run-script/lib/no
de-gyp-bin:/home/user/.sdkman/candidates/jbang/current/bin:/home/user/.sdkman/c
andidates/java/current/bin:/home/user/.nvm/versions/node/v22.17.1/bin:/usr/local/bi
n:/usr/bin:/bin:/usr/local/games:/usr/games\",\"SHELL\":\"/bin/bash\",\"TERM\":\"xte
rm-256color\",\"USER\":\"user\"}","transportType":"stdio"}
```

```
STDIO transport: command=/home/user/.sdkman/candidates/jbang/current/bin/jbang,
args=--quiet,org.acme:code-with-quarkus:1.0.0-SNAPSHOT:runner
Created server transport
Created client transport
Received POST message for sessionId 34603bde-706d-4a6f-a08c-0ef204cf0004
Received POST message for sessionId 34603bde-706d-4a6f-a08c-0ef204cf0004
```

Next, click the List Tools button to check the list of tools exposed by the MCP server, as shown in Figure 17-14 and Figure 17-15.

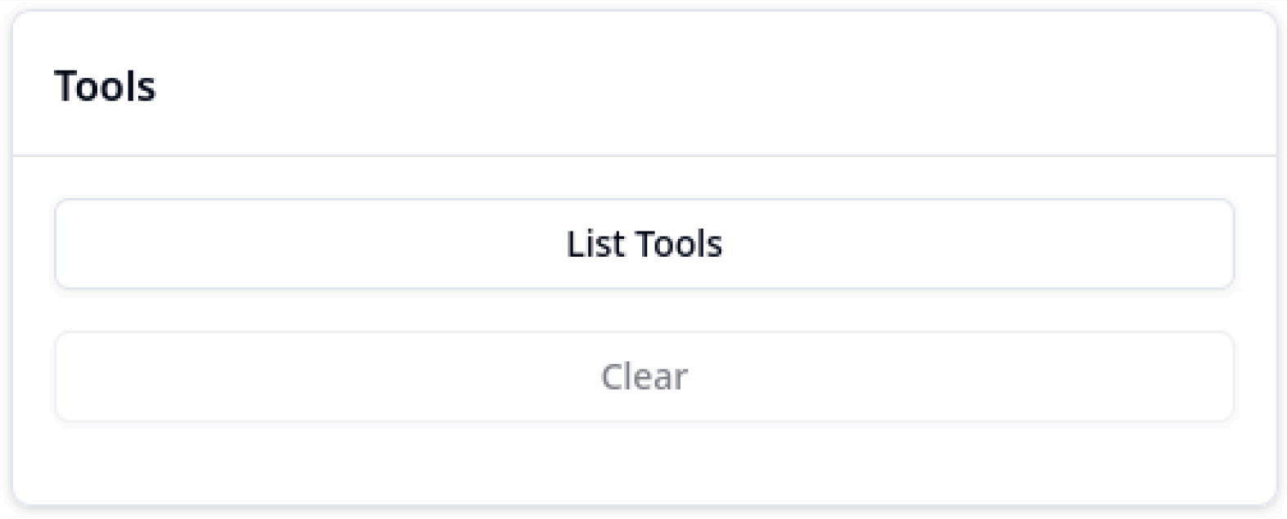


Figure 17-14 Tools exploration section of MCP Inspector

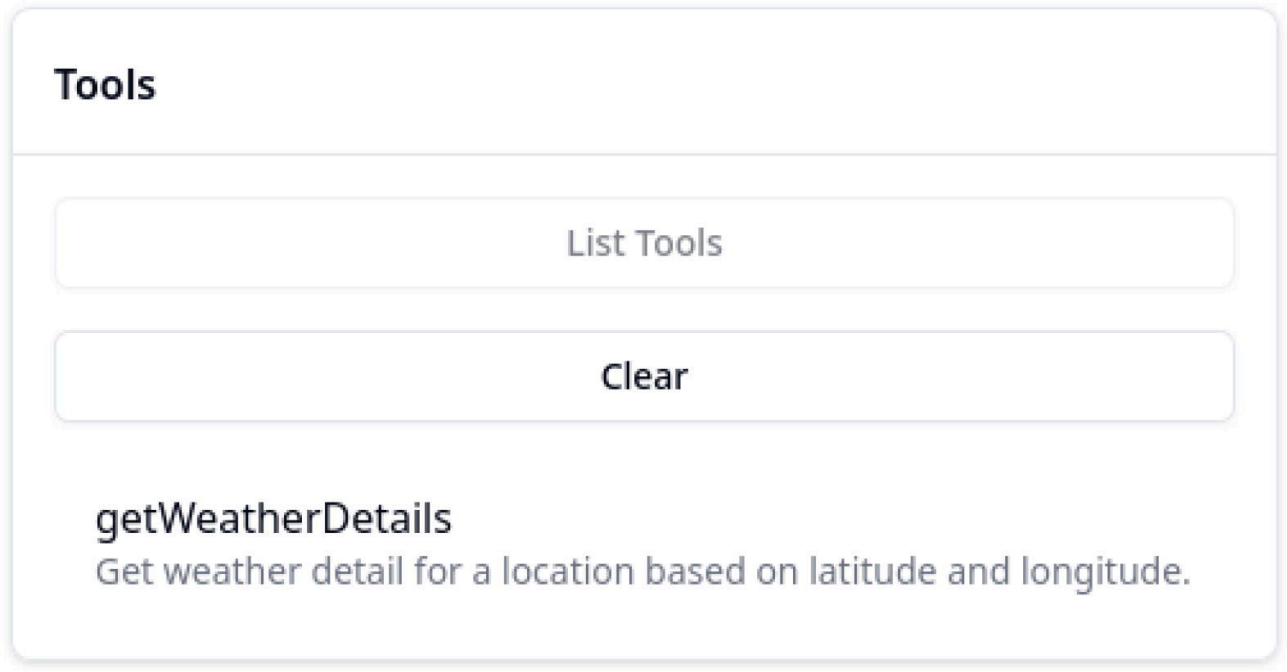


Figure 17-15 Tools listing from MCP Server using MCP Connector

The getWeatherDetails tool can be clicked. When clicked, it will show the details of the tools including the arguments it needs, as shown in Figure 17-16.

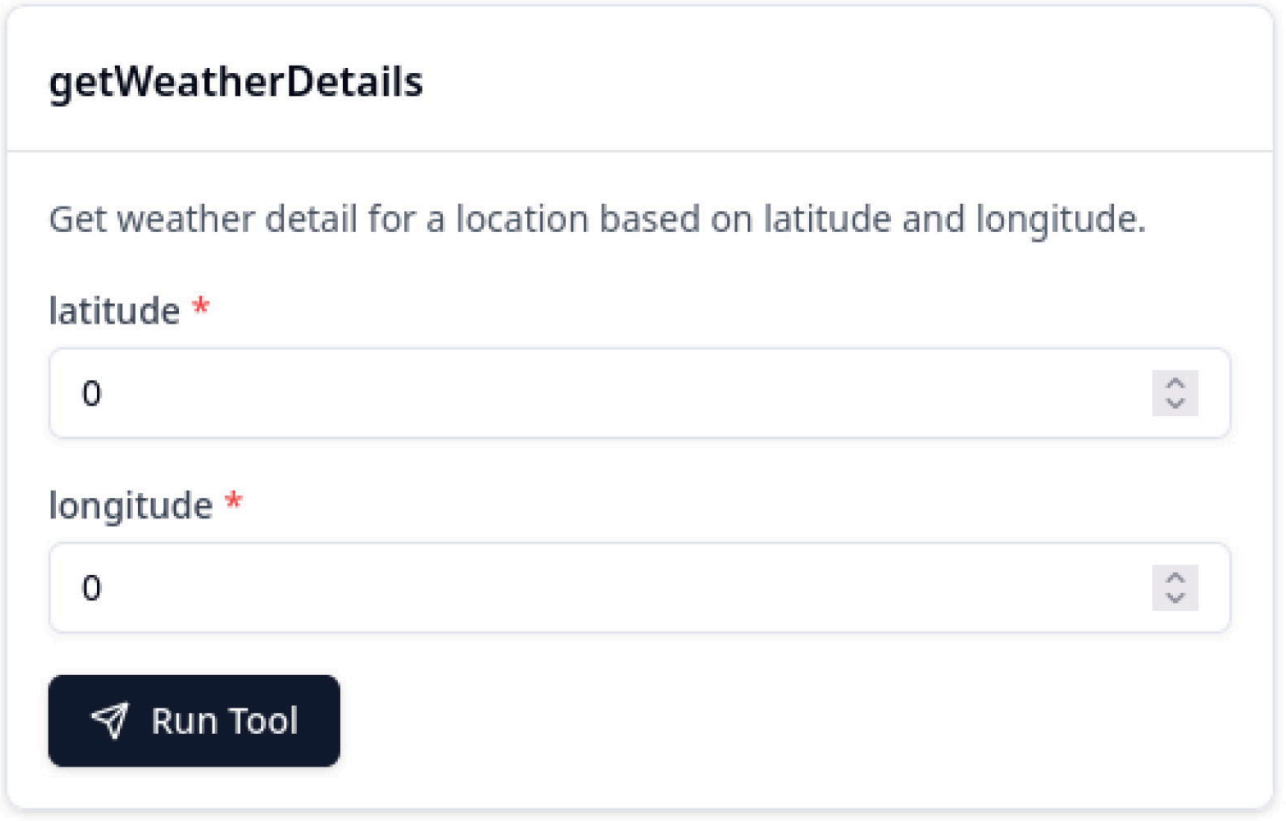


Figure 17-16 getWeatherDetails tool section in MCP Inspector

Let's provide a latitude and longitude and click the Run Tool button to see the output of the MCP server, as shown in Figure 17-17.

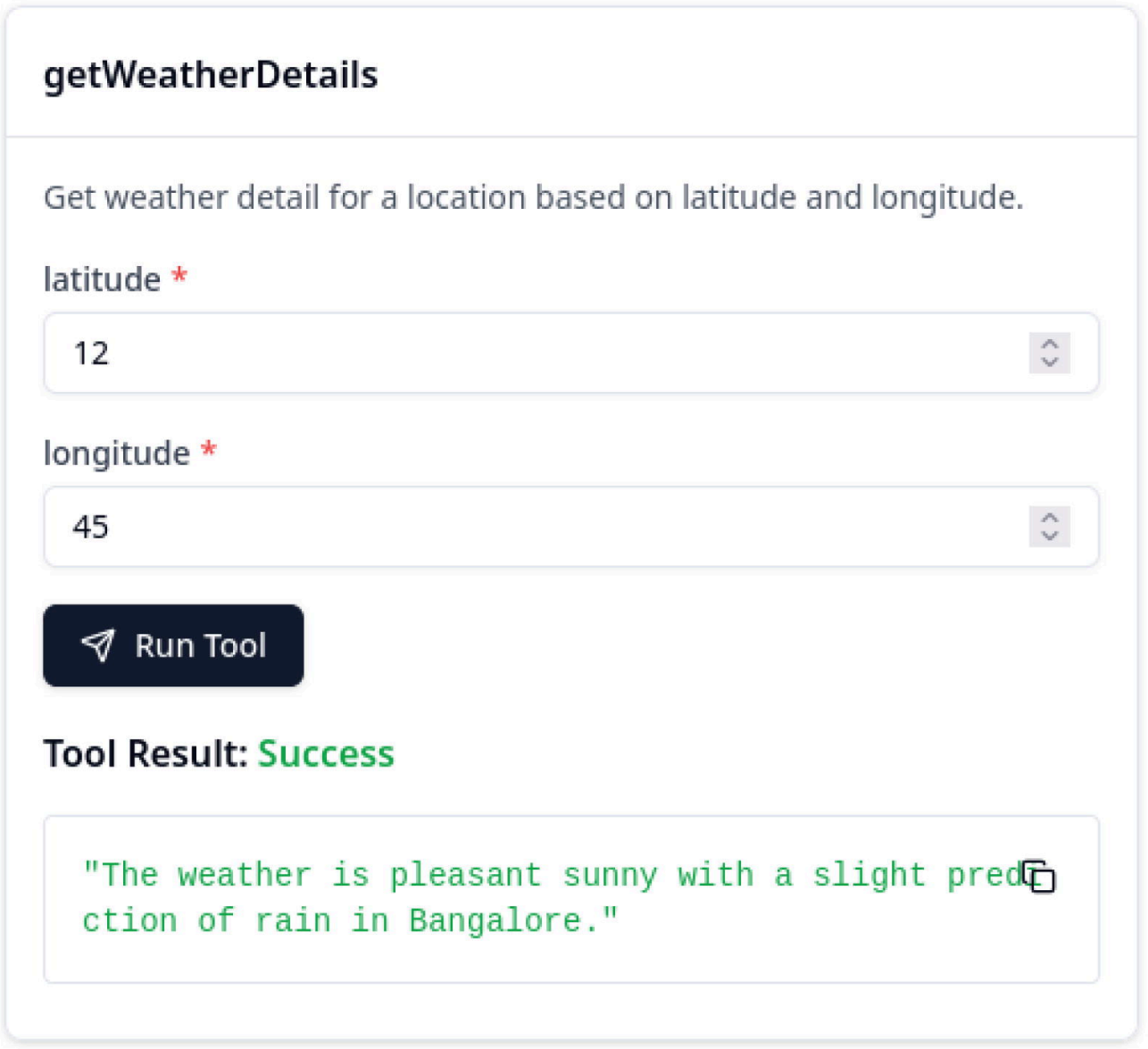


Figure 17-17 getWeatherDetails invocation from MCP Connector

This tool is pretty handy in helping to test and explore the MCP server and its various capabilities.

Conclusion

MCP addresses a critical need in the AI ecosystem: a standardized, structured, and modular way for agents to communicate with external tools and services. By abstracting tool interactions through well-defined message schemas and transport mechanisms, MCP reduces coupling, fosters reusability, and enhances system flexibility.

This chapter demonstrated how MCP not only unifies communication in theory but also translates into practice through robust implementations in Spring AI and Quarkus. From configuring tool exposure in an MCP server to invoking those tools from an MCP client, you learned how different teams or services can independently build, deploy, and consume AI capabilities in a microservice-like model.

As generative AI continues to shift from monolithic solutions to distributed, tool-augmented agent architectures, protocols like MCP become foundational. They not only enable current interoperability but also pave the way for future collaboration between LLM agents, services, and organizations. Embracing these standards will be key to unlocking the next generation of scalable, intelligent, and maintainable AI-driven applications.

The next chapter covers building observability into our applications to measure LLMs.

Previous chapter
< 16. Cloud Power-Ups: Bedrock, Vertex, and Azure OpenAI

Next chapter
18. Can You See Me Now? Observability for LLM Pipelines >